

به نام خدا

این پروژه برای پازل 8 هست .

این پازل در صفحه 3×3 و یک خانه خالی است . هدف این است که با جابه جایی خانه خالی، پازل را از حالت اولیه به حالت هدف برسانیم. هر حرکت هزینه برابر 1 دارد. عدد 0 همان جای خالی است. حالت هدف به صورت زیر هست:

1 2 3
4 5 6
7 8 0

در این کد حال هدف را به صورت زیر داریم:

GOAL_STATE = (1,2,3,4,5,6,7,8,0)

در این پروژه از سه الگوریتم جستجوی آگاهانه استفاده شده عبارتند از: A^* و IDA^* و RBFS

در جواب زمان اجرا ، مصرف حافظه و تعداد گره های گسترش یافته وجود دارد.

برنامه پایتونی که نوشتیم از روش های زیر استفاده میکند تا پازل را وارد کند :

1. یک پازل از قبل مشخص شده ، 2. وارد کردن یک پازل دلخواه توسط کیبرد ، 3. به هم زدن یک پازل مرتب شده به تعداد دلخواهی که توسط کاربر تعیین می شود.

کاربر میتواند هر زمان که بخواهد از برنامه خارج شود با یکی از کلمات زیر

"exit" / "quit" / "q" / "x"

این کد چک میکند آیا ورودی قابل حل هست یا خیر که تابع `is_solvable` تعداد وارونگی ها را می شمارد. در پازل 8 اگر تعداد وارونگی زوج باشد یعنی مسئله قابل حل است.

برای هیوریستیک های استفاده شده دو تابع تخمین هزینه استفاده شده است :

1. فاصله منتهن: مجموع فاصله افقی و عمودی هر عدد تا جایگاه هدف. (دقیق و سریع)

2. فاصله اقلیدسی: فاصله مستقیم (ریشه دوم مجموع مربع اختلاف ها) که کمی ضعیف تر منتهن هست.

الگوریتم های جست و جو:

1. الگوریتم A^* :

از $f(n) = g(n) + h(n)$ استفاده میکند که $g(n)$ هزینه طی شده و $h(n)$ تخمین باقی مانده از مسیر هست.

درکد از از heapq برای صف اولویت استفاده شده - closed set برای جلوگیری از تکرار - g_cost برای نگه داشتن بهترین هزینه رسیدن هست. این روش سریع هست و جواب بهینه رو تضمین میکند. ولی مصرف زیادی از حافظه رو هم دارد.

2. الگوریتم IDA^* :

IDA^* (Iterative Deepening A^*)

این همان A^* با حافظه کم هست. به جای نگه داشتن لیست باز بزرگ، یک مقدار bound تعیین می کند، فقط گرهایی با $f \leq$ bound بررسی می شوند و اگر پیدا نشد، bound افزایش می یابد.

درکد از جستجو به صورت DFS بازگشتی استفاده میشود - فقط مسیر جاری در حافظه است - از path برای جلوگیری از حلقه استفاده شده است. خوبی این الگوریتم این هست که مصرف حافظه کمی دارد ولی تکرار گره ها زیاد هست و کندتر از A^* هست.

3. الگوریتم RBFS :

RBFS (Recursive Best-First Search)

این الگوریتم مثل A^* بهترین مسیر را دنبال می کند اما فقط یک مسیر را در حافظه نگه می دارد.

درکد هر نود شامل g و f است و مرتب سازی successors بر اساس f کار میکند و کنترل با f_limit هست. خوبی ان این هست که مصرف حافظه کمی دارد ولی کمتر از IDA^* تکرار می کند. بدی که دارد این هست که پیچیده هست و ممکن است بازگشت های زیاد داشته باشد.

بهینگی	حافظه	سرعت	الگوریتم
بله	زیاد	زیاد	A^*
بله	کم	متوسط	IDA^*
بله	کم	متوسط	RBFS

در آخر برای اندازه گیری در تابع `measure_and_solve` ، زمان اجرا را اندازه می گیرید ، مصرف حافظه را محاسبه می کند و آمار زیر را گزارش می دهد:

- Path to goal
- Cost of path
- Nodes expanded
- Search depth
- Max search depth
- Running time
- Memory usage

پس از حل در خروجی ، آمار کامل الگوریتم را نمایش می دهد و مسیر حرکت را مرحله به مرحله چاپ می کند که شکل گرافیکی ساده از پازل نشان می دهد.

توضیح خلاصه درمورد هر تابع :

`safe_input`

دریافت ورودی از کاربر با امکان خروج امن از برنامه در صورت وارد کردن کلمات خروج یا بروز خطا.

`print_puzzle_grid`

نمایش وضعیت پازل به صورت جدول 3x3 متنی برای فهم بهتر حالت فعلی.

`manhattan_distance`

محاسبه فاصله منتهن بین دو مختصات روی صفحه (جمع اختلاف سطر و ستون).

`euclidean_distance`

محاسبه فاصله اقلیدسی بین دو مختصات با استفاده از فرمول ریشه دوم مجموع مربع اختلاف ها.

`heuristic_manhattan`

محاسبه مجموع فاصله منتهن تمام کاشی ها تا موقعیت هدف به عنوان تابع هیوریستیک.

`heuristic_euclidean`

محاسبه مجموع فاصله اقلیدسی کاشی ها تا موقعیت هدف به عنوان تابع هیوریستیک.

`is_solvable`

بررسی قابل حل بودن پازل با شمردن تعداد وارونگی ها و چک کردن زوج بودن آن.

`get_neighbors`

تولید تمام حالت های ممکن بعدی با حرکت دادن خانه خالی به بالا، پایین، چپ یا راست.

reconstruct_path

بازسازی مسیر نهایی از حالت شروع تا هدف با استفاده از اطلاعات والدها.

a_star_search

پیاده‌سازی الگوریتم A^* برای یافتن کوتاه‌ترین مسیر با استفاده از $g+h$ و صف اولویت.

ida_star_search

پیاده‌سازی الگوریتم IDA^* با جستجوی عمقی تکرارشونده و محدودیت روی مقدار f .

infer_action

تشخیص نوع حرکت انجام‌شده بین دو وضعیت متوالی برای بازسازی مسیر در IDA^* .

rbfs_search

پیاده‌سازی الگوریتم RBFS که بهترین مسیر را به صورت بازگشتی و با حافظه کم دنبال می‌کند.

measure_and_solve

انتخاب الگوریتم و هیوریستیک، اجرای حل مسئله و اندازه‌گیری زمان و حافظه مصرفی.

get_predefined_puzzle

نمایش چند پازل آماده و دریافت انتخاب کاربر.

get_manual_puzzle

دریافت ورودی دستی پازل از کاربر و بررسی صحت اعداد وارد شده.

shuffle_from_goal

ایجاد پازل تصادفی با انجام تعدادی حرکت معتبر از حالت حل‌شده.

get_shuffled_puzzle

دریافت تعداد حرکات تصادفی از کاربر و تولید یک پازل قابل حل.

choose_heuristic

دریافت انتخاب کاربر برای نوع تابع هیوریستیک (Manhattan یا Euclidean).

choose_algorithm

دریافت انتخاب کاربر برای نوع الگوریتم جستجو (A^* ، IDA^* ، RBFS).

show_solution_path

نمایش مرحله‌به‌مرحله مسیر حل از حالت اولیه تا هدف.

print_result_summary

چاپ خلاصه آماری شامل هزینه مسیر، تعداد گره‌ها، عمق جستجو، زمان و حافظه.

main

تابع اصلی برنامه که جریان کلی اجرا، دریافت ورودی، حل مسئله و تکرار برنامه را کنترل می‌کند.

تحلیل پیچیدگی همین کد:

الگوریتم A^* :

زمان: در بدترین حالت نمایی $O(b^d)$ چون تمام گره‌های ممکن تا عمق جواب بررسی می‌شوند.
حافظه: نمایی $O(b^d)$ چون open list ، closed set و g_cost همه در حافظه نگهداری می‌شوند.

الگوریتم IDA^* :

زمان: نمایی $O(b^d)$ و معمولاً بیشتر از A^* چون گره‌ها در هر افزایش bound دوباره تولید می‌شوند.
حافظه: خطی $O(d)$ چون فقط مسیر جاری (stack بازگشتی) در حافظه نگهداری می‌شود.

الگوریتم RBFS :

زمان: نمایی $O(b^d)$ مشابه A^* ولی ممکن است به علت بازگشت‌های مکرر کندتر شود.
حافظه: خطی $O(d)$ چون فقط مسیر فعلی و تعداد کمی از جانشین‌ها نگهداری می‌شوند.

در این کد: b حداکثر 4 که حرکت‌های ممکن خانه خالی هست و d عمق جواب که تعداد حرکت تا رسیدن به هدف هست.

در نهایت میتوان نتیجه گرفت که A^* سریع‌تر ولی پرمصرف‌تر از نظر حافظه هست و IDA^* و RBFS کم‌حافظه‌تر ولی معمولاً کندتر از A^* هستند.